

Subject:

Object Oriented Programming with Java

BCA 5th Semester

Course Code: BCA502

Comprehensive notes provided by

Great Heights

Build | Believe | Become



Introduction to Java

Java was developed by Sun Microsystems Inc in 1991, later acquired by Oracle Corporation (Owner at present). It was developed by James Gosling, Mike Sheridan, and Patrick Naughton. Writing, compiling and debugging a program is easy in java. It helps to create modular programs and reusable code. Java was designed with the philosophy: – “Write Once Read Anywhere”. This means a Java program written on one operating system can run on any other operating system, provided that a JVM is installed.

The language was first called Greentalk, then renamed Oak, inspired by an oak tree outside James Gosling’s office. Since the name Oak was already trademarked, the team later selected the name Java after brainstorming names like Silk, Lyric, DNA, and Dynamic.

First official release of Java: JDK 1.0 (Year 1996)

Applications of Java

Desktop Applications: Java can create desktop applications with graphical user interfaces (GUIs).

Best Java Framework for Desktop Applications

- Spring Boot
- JavaFX – The Versatile Choice
- Swing – A Time-Tested Classic
- Apache Pivot – Focused on Productivity
- AWT (Abstract Window Toolkit) – The Foundation
- Vaadin – Empowering Web Developers
- SWT (Standard Widget Toolkit) – Native Look and Feel
- Java SWT/JFace and Swing Integration – Expanding Possibilities
- Griffon – Embracing Convention over Configuration
- Java Native Access (JNA) – Bridging Native Libraries

Web Applications: Java Programming is used for server-side web application development, providing scalable and robust solutions. Java language is used in many government, healthcare, insurance, education, and defense organizations to develop online applications and deliver digital services. Google’s Gmail is a prime example of a Java web application.

Mobile Applications: Java application development has been growing for a long time because this powerful language allows developers to create secure, robust and feature-rich mobile products and digital services. Android applications are mainly developed in Java and Kotlin.

Enterprise Applications: Java is used in many areas across large enterprise organizations.

- Web and application servers
- ERP and CRM
- Financial systems

Cloud-Based Applications: Java is platform-independent; it provides a comprehensive approach to cloud-based applications. Java is used to develop cloud-based applications, leveraging various cloud platforms. Java’s compatibility with popular cloud service providers like AWS, Google Cloud Platform, Microsoft Azure, etc. increases the attractiveness of cloud-based development.

Game Development: Java can be used for developing games, especially for platforms where performance requirements are less stringent. Example – Minecraft.

Features of Java

Java's extensive standard library and rich ecosystem of frameworks, libraries, and tools further bolster its popularity. The language is widely used in various domains, from enterprise-level applications and mobile development to web applications and backend systems. Additionally, Java's support for multithreading allows developers to create efficient, responsive, and concurrent applications, a crucial feature in the era of modern computing.

Furthermore, Java's active community and continuous updates ensure that it stays relevant and aligned with evolving industry needs. With its versatility, reliability, and widespread usage in critical industries like finance, healthcare, and telecommunications, Java continues to be a cornerstone in the world of software development, attracting both seasoned professionals and those entering the field.

1. Simple: Java's syntax is designed to be simple and easy to understand. With its similarity to other programming languages like C++ and C, many web developers find it accessible, easing the learning curve. Unlike C++, Java removes pointer arithmetic, multiple inheritance using classes, operator overloading.

2. Platform-independent: One of the main features of Java is its platform independence. The "write once, run anywhere" principle enables Java code to be compiled into an intermediate form (bytecode) that can run on any device with a Java Virtual Machine (JVM). This platform independence enables developers to create applications that can run seamlessly on diverse platforms without modification.

3. Object-oriented: Java is fully object-oriented, emphasizing the use of classes and objects. This approach promotes modularity, reusability, and a clear structure in code organization. For example, in Java, a developer can create a "Car" class with attributes like "model" and "color" and instantiate multiple objects representing different cars.

4. Robust and Secure: Java's design prioritizes reliability and robustness. It includes features like automatic garbage collection, exception handling, and strong memory management, reducing the chances of system crashes. The security features in Java, such as the absence of explicit pointers and a secure runtime environment, contribute to its reputation for building secure applications.

5. Multithreading: Java supports multithreading, allowing concurrent execution of multiple threads within a program. This feature enhances performance, responsiveness, and the efficient utilization of system resources. Developers can use Java's `Thread` class or implement the `Runnable` interface to create multithreaded applications.

6. Distributed Computing: Java is well-suited for distributed computing. Its Remote Method Invocation (RMI) and Application Programming Interface (API) for socket programming simplify the development of distributed applications.

7. Dynamic and Extensible: Java's dynamic nature allows developers to adapt and modify the code during runtime. This extensibility contributes to the creation of scalable and adaptable software.

8. Portable: Java's portability stems from its ability to run on any system with a Java Virtual Machine (JVM). This feature comes from the platform-independent bytecode that Java programs turn into when compiled. This bytecode can then work on any machine with a JVM, without needing changes to the code.

9. Architecture-neutral: Java has an architecture-neutral design. This means that once compiled, the code (bytecode) can work on any type of processor. Java achieves this by leaving out features specific to certain architectures in both its language and runtime. As a result, Java works the same way across different hardware setups.

10. Interpreted: Java compiles programs into bytecode. The JVM then interprets and runs this bytecode. This gives Java a lot of flexibility. The same compiled code can run on any device with a JVM, no matter what the underlying architecture is.

11. High Performance: Java is an interpreted language. Yet, it delivers strong performance through Just-In-Time (JIT) compilation. The JVM turns bytecode into native machine code as the program runs. This boosts the performance of code paths that run often.

Setting up Java Environment

Step 1: Download JDK

Visit the Oracle Java Downloads page or choose an open-source version like OpenJDK.

Step 2: Install JDK

Download the installer compatible with your operating system (Windows, macOS, or Linux).

Run the installer and follow the on-screen prompts. Copy the installation path where Java was saved (e.g., C:\Program Files\Java\jdk-25 on Windows or /Library/Java/JavaVirtualMachines/ on macOS)

Step 3: Configure Environment Variables

For Windows:

Press Win + S, search for "Environment Variables", and select Edit the system environment variables.

Click the Environment Variables button at the bottom of the window. Under System Variables, click New. Set the Variable name to JAVA_HOME and paste your JDK installation path into the Variable value field. Click OK.

In the same System Variables section, look for the variable named Path (or PATH), select it, and click Edit. Click New and type %JAVA_HOME%\bin.

Click OK on all open windows to save the changes

Step 4: Verify the Setup

Open a new Command Prompt or Terminal window and run the following commands to confirm your installation is working:

Check the compiler version: `javac -version`

Check the runtime version: `java -version`

PATH and CLASSPATH Variables

In Java, PATH and CLASSPATH are distinct environment variables used for entirely different purposes: PATH is used by the operating system to find Java development tools (java, javac), while CLASSPATH is used by the Java Virtual Machine (JVM) and compiler to locate compiled bytecode (.class or .jar files)

PATH: The PATH variable provides a list of directories the OS searches to find execution commands. Without setting PATH, one would have to type out the absolute path (e.g., C:\Program Files\Java\jdk-xx\bin\javac MyProgram.java) every single time they want to compile a file. To configure it, it needs to be directly pointed to the JDK installation's bin directory.

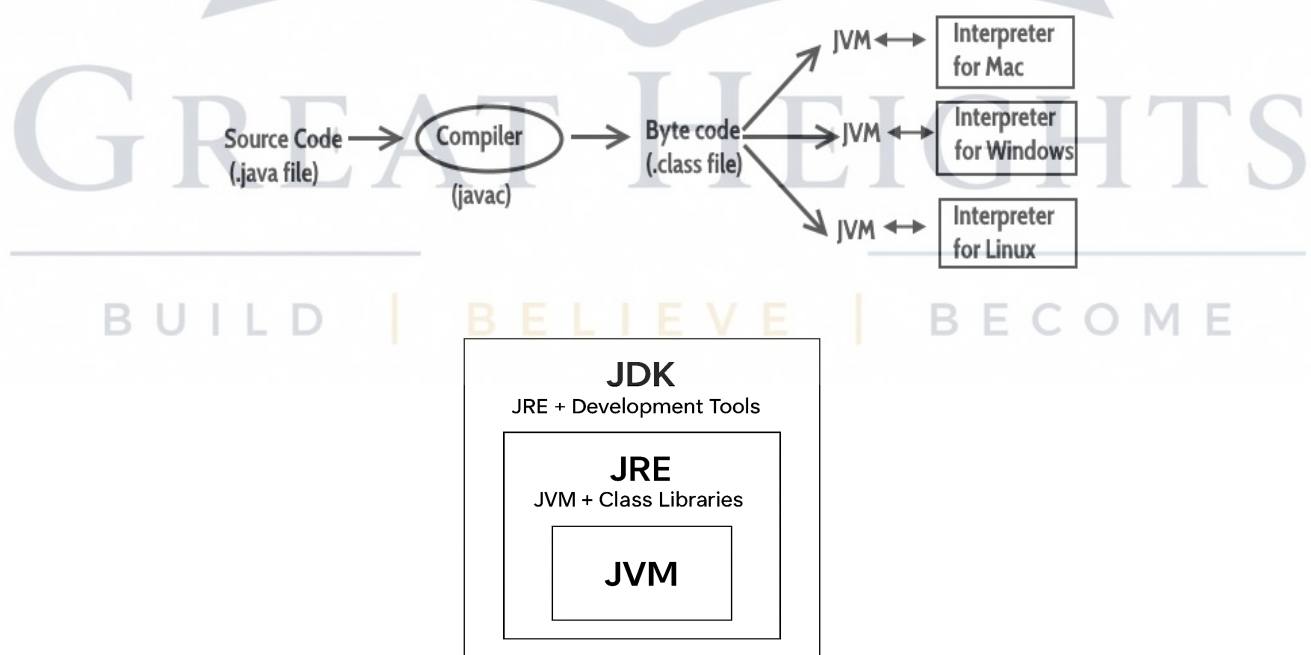
CLASSPATH: Once the operating system successfully starts the Java compiler or the JVM, Java takes over. If the program relies on external libraries, third-party databases, or files located in separate packages, the Java ClassLoader needs to know where to find those compiled .class and .jar components. If the application depends on a library located somewhere else on the machine, Java will throw a `NoClassDefFoundError` or `ClassNotFoundException` unless that location is registered on the classpath.

Feature	PATH Variable	CLASSPATH Variable
Primary User	Operating System (OS)	Java Compiler and JVM
Purpose	Finds system executable tools (.exe or binaries)	Finds compiled Java bytecode (.class , .jar)
Typical Target	The JDK binary directory (/bin)	Application directories or dependency libraries
Scope	System-wide (affects all CLI commands)	Java-specific (affects package resolution)

Java Virtual Machine (JVM)

Java Virtual Machine (JVM) is a virtual machine that resides in the real machine (any computer) and the machine language for JVM is byte code. This makes it easier for the compiler as it has to generate byte code for JVM rather than different machine code for each type of machine. JVM executes the byte code generated by the compiler and produces output. JVM is the one that makes the java platform independent.

Java source code is compiled into bytecode by Java compiler. The JVM reads the byte code and converts it into machine code that the operating system can execute.



JDK (Java Development Kit): JDK is a software development kit used to build Java applications. It contains the JRE and a set of development tools.

- Includes compiler (javac), debugger, and utilities like jar and javadoc.
- Also includes the JRE, so it also allows running Java programs.
- Required by developers to write, compile, and debug code.

JRE (Java Runtime Environment): JRE provides an environment to run Java programs but does not include development tools. It is intended for end-users who only need to execute applications.

- Contains the JVM and standard class libraries.
- Provides all runtime requirements for Java applications.
- Does not support compilation or debugging.

Note:

- JRE is only for running applications, not for developing them.
- It is platform-dependent (different builds for different OS).

JVM (Java Virtual Machine): JVM is the core execution engine of Java. It is responsible for converting bytecode into machine-specific instructions.

- Part of both JDK and JRE.
- Performs memory management and garbage collection.
- Provides portability by executing the same bytecode on different platforms.

Note:

- JVM implementations are platform-dependent.
- Bytecode is platform-independent and can run on any JVM.
- Modern JVMs rely heavily on Just-In-Time (JIT) compilation for performance.

1. Class Loader Subsystem

Loads class files into memory.

Works in three steps: Loading, Linking, and Initialization.

- Loading: Loads classes using Bootstrap, Extension and Application Class Loaders.
- Linking: Verifies the bytecode, allocates memory and resolves references.
- Initialization: Initializes static variables and executes static blocks.

Example: When java MyApp is run, the Class Loader loads MyApp.class into memory.

2. Runtime Data Areas

The JVM divides memory into several regions:

- **Method Area**: Stores class-level data like metadata, static variables, and method code.
- **Heap**: Stores objects created using new.
- **Stack**: Each thread gets its own stack storing method frames (local variables, partial results).
- **PC Register**: Keeps track of the current instruction.
- **Native Method Stack**: For methods written in other languages (like C).

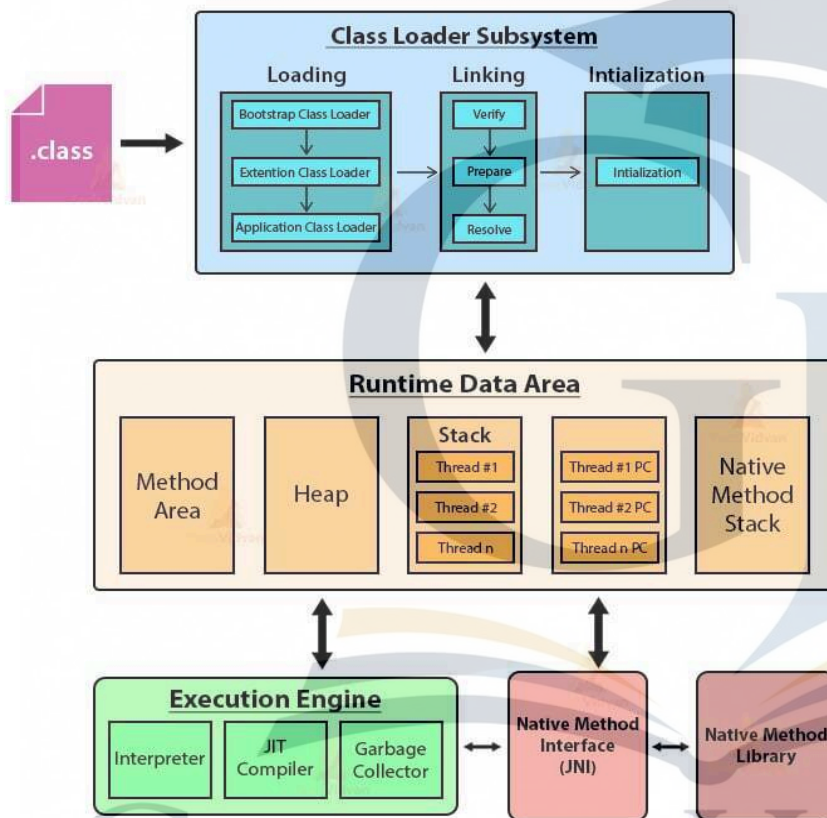
3. Execution Engine

- Interpreter: Reads and executes bytecode instructions one by one.
- JIT Compiler: Converts frequently used bytecode into native machine code for better performance.
- Garbage Collector: Frees unused objects from the heap to reclaim memory.

4. Native Method Interface (JNI)

This acts as a bridge between Java code and native libraries (e.g., calling C libraries from Java).

JVM Model



Why we need JRE:

- Programs built on Java (like legacy enterprise tools, older versions of Minecraft, or specific banking apps) require it.
- Running Server Applications: Production servers that only execute ready-made corporate software utilize the JRE to save system space.
- Executing Jar Files: Standard .jar files require the JRE environment to load, verify, and run bytecode.